

Chapter 17

CS2 course package - Casper College

Charlotte Gruner¹

¹Casper College

Abstract

|Your abstract text here.|

Descriptor Elements

Programming Language Employed: primarily C++, Java (for Greenfoot project)

Textbook used: REVEL edition of Starting out with C++ by Tony Gaddis[?]] (Spring 2026)

Relevant core courses: CS2 (COSC 2030)

Pre-requisites: CS1 (COSC 1010)

Relevant PDC topics:

Relevant PDC topics and Blooms levels

PDC Concept	Chapter Section					
	17.3.1, 17.4.1	17.3.2	17.4.2	17.3.3	17.4.3	17.4.4
PDC Vocabulary and Foundations	RE,UN					
Shared Resources and Race Condition		UN	AP			
Performance Analysis			AN			
Event Driven Programming				UN,AP		
Task Parallelism						UN, AP
Performance Benchmarking	AP		AP			

Learning outcomes:

1. Define and recognize fundamental PDC terms such as multi-core, data parallelization, and task parallelization.
2. Apply OpenMP directives to parallelize array processing and analyze the resulting performance gains through benchmarking.
3. Identify and explain race conditions and shared resource conditions in both physical simulations and C++ code.
4. Create a novel interactive 2-D game using the Greenfoot API to demonstrate mastery of event-driven programming and object-oriented design.

Context for use: Casper College is a 2-year public college located in central Wyoming. The total enrollment at Casper College is typically between 4500 students and 5000 students. The most popular programs are in general studies, health care, and agriculture fields. The Computer Science Department, which is housed within the Mathematics Department in the School of Science generally consists of around 50 declared majors with 80% working on freshman level coursework and 20% working on sophomore level courses. Demographically, Casper College has more students who are older than the

typical 18-21 age group found at a typical university in the United States and less than half of the students are full time. Casper College also attract local high school students who have the opportunity to take College Classes for both high school and college credit both at their local high schools through concurrent enrollment and on the Casper College campus through dual enrollment. The Computer Science Department has one full time faculty member. The Computer Science Associates Degree at Casper College is meant to serve as the first two years of a 4-year degree program and includes the following core computer science classes: CS0 (COSC 1010) Introduction to Computer Science, CS1 (COSC 1030) Computer Science I, CS2 (COSC 2030) Computer Science II, (COSC 2150) Computer Organization, (COSC 2300) Discrete Structures. Class sizes for 1000-level courses tend to be below 20. Class sizes for 2000-level courses tend to be under 10. At Casper College the CS2 course, COSC 2030 is considered a 3 hour lecture, 2 hour lab class, but it is currently taught in two weekly class meetings of 135 minutes each.

17.1 Introduction

While it is generally recognized that introducing Parallel Distributed Computing (PDC) content to computer science students earlier and with more regularity particularly in the first two years of the computer science core classes would be advantageous, how to do this without sacrificing coverage of traditional topics is not clear. This chapter discusses the attempt to include curricular elements adapted from the Development Teams' curricula to introduce PDC topics into the Casper College traditional CS2 in the Fall of 2025 course in a meaningful way for students without sacrificing core CS2 topics. To meet this objective, I focused on unplugged activities and some simple coding exercises that would introduce PDC vocabulary with in-class work that I could supervise and guide. This class is taught in two 135 minute sessions, so I felt there would be adequate time to still cover all the material that I normally covered in class. In addition, the normal material had both a text book and often my video content. Since the PDC material was new and this was the first semester that I had taught it, I wanted to take a more hands-on approach. The exception to this approach was the inclusion of the Greenfoot project materials, which was added as a midterm project for the students. In this case, I felt that students would be able to watch the tutorial videos and make progress on their own.

17.2 How CS2 was changed and PDC topics integrated

I opted to introduce some aspect of PDC every week for weeks 1-5 in either an in class unplugged activity, in class plugged in activity, or a short homework assignment. During these weeks we were also covering the same material that is normally covered in the class with only minimal adjustments. These first 5 weeks included three short unplugged activities and an introduction to OpenMP. In weeks 6-9, students were assigned a midterm project using Greenfoot. This serves as an introduction to event handling, a core PDC topic. In the second half of the semester, the students did one more in-class activity using OpenMP to parallelize a version of QuickSort during a discussion of sorting algorithms.

17.2.1 Integrated Syllabi (Pre and Post)

Week	Pre (Baseline Fall 2024)	Post (PDC content added Fall 2025)
1	Review pointers/classes	Added penny search unplugged as icebreaker
2	C-strings, string class	Added in class intro to OpenMP parallelizing array processing
3	Classes (instance, static members, friends, member-wise assignment, copy constructors)	OpenMP and array accumulation in class
4	Classes (operator overloading)	Unplugged shared resource demonstration
5	Classes (object conversion, Aggregation, Rvalue References and Move Semantics)	Use OpenMP to solve the problem with shared resources in week 3 code, fork join activity
6	Classes (inheritance)	Intro to Greenfoot
7	Classes (polymorphism, abstract base class)	Greenfoot project
8	Templates	Greenfoot project
9	STL Library	Greenfoot project presentations
10	STL	Final Project - game engine
11	Linked Lists	—
12	Stacks & Queues	—
13	Recursion	no new content
14	Binary Trees	parallel implementation of quicksort
15	Searching, Sorting, Big O, AVL Trees, Heaps, Hashes	no new content

17.2.2 Highlights

The most substantial change to the syllabus from Fall 2024 to Fall 2025 was the inclusion of the Greenfoot-based midterm project. Both sections of the class covered the same core CS2 material.

17.2.3 Challenges

The greatest pedagogical hurdle with the modules added was the C++ coding exercises using OpenMP. Even greatly simplified, the use of OpenMP and the chrono libraries has high intrinsic cognitive load and requires significant time for students to understand how to use these libraries.

17.3 New Unplugged Activities

17.3.1 Unplugged Penny Search

This activity is described in 1.3.2. This activity was also used at Casper College in COSC 1030, the CS1 course in Spring semester of 2025. 7.3.2.

Problem Description, Prerequisites, and Learning Outcomes

Problem Description This activity is described in detail in section 1.3.2.

Implementation Details

. I adapted April Crockett’s materials described in section 2.3 for use in my own classroom and as an guided at home activity for on-line students.

Prerequisites. No prerequisites are required for this activity.

Learning Outcomes.

- **Logistics (Material required for the activity, etc.)**
- **How to Conduct**
- **Instructional materiel (Lectures, Videos, Assignments, etc.)**

Experience and Teaching Tips

I used this activity in week 1 of instruction for COSC 2030 (CS2) in the Fall of 2026. This group had not had any PDC material presented to them in class since they took COSC 1010 (CS1) in the Spring semester of 2026, which was during the baseline year for my portion of the research. This unplugged activity was highly engaging to this small group of six students. Because the group was small, we never really got to the point of diminishing returns with adding enough students to the group to the point that performance decreases, but students could see that this would happen eventually.

Data and Analysis

17.3.2 Unplugged Coin addition

Problem Description, Prerequisites, and Learning Outcomes

Problem Description: This unplugged activity simulates the summation of all the elements in an array with a shared accumulator. In this activity students are tasked with adding the value of a cup of coins together to find the total monetary value. This activity is similar to the Penny Search Sort activity in that students will recognize that this task can be done faster if divided between students. But the accumulated value is stored by the instructor. To add to the accumulated value, students must first request the current value of the accumulator, then add their current coin, then update the accumulator. The process of multiple students attempting to access and update the accumulator creates a race condition.

Prerequisites: The ideal prerequisite for this activity is the Penny Search Sort activity. But this activity could easily be done without that prerequisite. Students should also have familiarity with arrays. It might also be useful for students to have attempted to parallelize a for loop using OpenMP to sum an array without using a parallel region to observe that they are not getting the correct answer.

Learning Outcomes: Students will learn to recognize a shared resource and a race condition in the context of PDC.

Implementation Details

- **Logistics (Material required for the activity, etc.)** Coins (possibly of different denominations) 5-10 coins per student should be sufficient. It is helpful to have cups to put the coins into.
- **How to Conduct** Students should be in groups of 5-6. Explain that one student (or the instructor in a small class) will be the memory location that will house the accumulator. The rest of the students should each get a cup of coins. Each of these students should take one coin out of the cup and attempt to add the value of that coin to the accumulator, this is the implementation of the line of code `total += coinval`. In order to do this operation, the student needs to request the current value of the accumulator by requesting (by email, chat, verbally, or passing a note), add the `coinval` to it and then update the value stored in the accumulator by sending it (by email, chat, verbally, or passing a note, etc.). Ideally, the student or instructor acting as the keeper of the accumulated total should have some sort of record of these transactions to show at the end.
- **Instructional material (Lectures, Videos, Assignments, etc.)** The follow-up lecture involves using OpenMP to implement a solution.

Experience and Teaching Tips

This activity was used with the CS1 course at Casper College in Spring 2026 as well as the CS2 Course the semester before in Fall 2025. Both groups were small and I acted as an accumulator. Although I thought I had explained the directions, some students definitely did not get the directions correct. In both cases after we ran the experiment once, we ended up doing it again and the light bulb seemed to go on for most students in terms of recognizing that having a shared resource in a parallel program can have problems. I don't have a good solution for on-line/asynchronous students to be able to participate in this exercise, but it could be done with on-line synchronous students if they could be warned to have 10 coins. Another problem is that students from other countries may not be familiar with U.S. currency.

Data and Analysis

17.3.3 Fork-Join Clicker Questions

Problem Description, Prerequisites, and Learning Outcomes

Problem Description This is a short activity, which was adapted from the KNOX materials (REFERENCE). I encouraged students to do this activity in pairs or small groups, students are given several precedence graphs and asked to figure out what the shortest number of time units that are required for two processors to complete the work and also the worst case number of time units, assuming no processor is voluntarily idle. A sample of one of the questions is shown in Figure 17.1. For this question, students should determine that

the worst case for two processors would be 5 time units and the best case would be 4 as shown in Figure 17.2.

For the following dependency graphs, assume each strand (box) runs in one time unit. Assuming there are 2 processors, what is the smallest number of time units that the computation could take. What is the maximum number of time units the computation could take. Justify your answer.

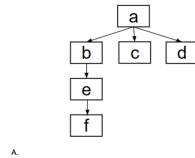


Figure 17.1: Description of the image content.

worst					
Wprocessor – time units>>	1	2	3	4	5
1	a	d	b	e	f
2	-	c	-	-	-

best				
Wprocessor – time units>>	1	2	3	4
1	a	b	e	f
2	-	c	d	-

Figure 17.2: Description of the image content.

Prerequisites Awareness of the existence of multi-processor CPU architecture. **Learning Outcomes**

- Students are introduced to the idea of separating a program into parts that can run on different cores.
- Students are introduced to the terminology of fork-join.
- Students will be able to analyze a dependency graph to determine the best and worst case scenarios for a 2 processor system.

Implementation Details

- **Logistics (materials required for the activity, etc.)** The materials needed are the handout for the students.
- **How to Conduct** These materials could be used as inclass work or as a homework assignment.
- **Instructional material (Lectures, Videos, Assignments, etc.)** The materials used for this activity are available at: https://github.com/charlottegruner/CDER_CC_CS2/upload/main/ForkJoinClickerQuestions

Experience and Teaching Tips

The students were given time in class (about 15 minutes) to work this assignment with their team. The students seemed to be engaged in the activity and it generated a good discussion. Six students in the class submitted work for this assignment. The hardest problem turned out to be B. Only one group correctly answered that the worst case scenario for B was 7 time units. The next time I use these materials, I will try to allow more time so that groups can compare answers to generate more discussion.

Data and Analysis

17.4 New Plugged-in Activities

17.4.1 Array Creation in parallel using OpenMP

Problem Description, Prerequisites, and Learning Outcomes

Problem Description In this activity students are asked to write a C++ program that allocates space for a large array and then fills the array with the same value as the array index. The students are shown how to use the pragma omp parallel for to parallelize the loop and are instructed to figure out how many different threads are created and which threads are handling which indices. The students are introduced to the use of the chrono library for timing code, so that the speedup can be documented. These materials were adapted from April Crockett's OpenMP materials (REFERENCE).

Prerequisites Experience with C++ arrays and dynamic memory allocation using pointers.

Learning Outcomes

- Students will define and explain parallel computing terms: parallel processing, sequential processing.
- Students will identify the number of cores on their particular machine.
- Students will learn to use the openMP pragma omp parallel for to do data parallelization
- Students will learn how to compile using the -fopenmp.
- Students will observe how the loop is split into chunks that are handled by different threads.
- Students will use the <chrono> library to measure and compare the execution time of sequential versus parallelized code.

Algorithms

Implementation Details

- **Instructional Material (Lectures, Videos, Assignments, Tests)** https://github.com/charlottegruner/CDER_CC_CS1/tree/main/ArrayCreation
OpenMP In these materials students are encouraged to work in pairs so that hopefully there will be some discussion that happens as they are working on this assignment. The first part of the assignment shown in Figure 17.3 asks the students to do some preliminary research on PDC related vocabulary and also some vocabulary specific to OpenMP to acquaint them with important terms. For this part of the assignment, students are allowed to use AI to ask for definitions.

Figure 17.3: Part 1 of Intro to OpenMP assignment

```
Part 1: Learn about the OpenMP API

OpenMP is an API that supports shared-memory parallel processing for C/C++ and Fortran.

1. Define API [AI]
2. Define parallel processing [AI]
3. Define shared-memory [AI]

OpenMP allows programmers to parallelize code by adding pragmas to existing code.

4. What's a pragma? [AI]
5. What is the goal of parallelization of code? [AI]
6. What's a "core"? [AI]
7. How many cores does the computer you are using have? Some ideas might be to find out what kind of processor your computer has (usually buried in a
   'settings' tab). Or if you are on a linux machine you can explore the commands lscpu, lscpu, and lscpu or lscpu. [AI]

The programming we have done so far in COSC 1010 and COSC 1030 has been sequential.

8. What does "sequential processing" mean? [AI]
```

In the second part of the assignment (Figure 17.4 and Figure 17.5, students are instructed to write a sequential program that should be straightforward, but should also take a measurable amount of time: constructing and initializing a large array. The next steps walk the students through thinking about whether or not this task could be done in parallel and how to implement this using the appropriate pragma from OpenMP. In this section, the students use a smaller array so that they can document via print statements which threads are performing which array index initializations, which should hopefully be the same as the number of cores that their computer has (which has been figured out in part 1).

Figure 17.4: Part 2 of Intro to OpenMP assignment

```
Part 2: Write a short program that will hopefully take a while if run sequentially. Then explore a few aspects of OpenMP to parallelize a section of the code.

In parallel computing, the goal is to split up the task into subtasks that can be performed by different processors. OpenMP is an API that allows a programmer to use
compiler directives to specify parts of the code that should run in parallel.

9. Write a program that allocates memory for 100 million integers and a loop that assigns each index in the loop to contain the integer corresponding to its index.
   So at index 0 the value in the array is 0, at index 1, the array is 1 and so forth. Note that you might have to explore the long and long long data types since
   the numbers we are dealing with here can be quite long. Feel free to use AI tools for this task to find out what the largest integer that can be stored on a given
   system. Remember that these values are not standard for C++, so you will need to figure out what those maximum values are on the particular system you are
   using. Also, remember that your system may not be able to allocate this much stack space, which will require using heap memory through dynamic memory
   allocation (using pointers). I encourage you to experiment with trying the program with stack memory to see what happens.

10. Once you have written the program in 10, consider the loop that fills the array. Is this an example of sequential processing or parallel processing?

11. Could this program be parallelized? Could tasks in this program be done by different cores at the same time? (Think back to the penny searching experience.)

12. The compiler directive to parallelize a loop is:



```
#pragma omp parallel for
```



The directive is placed directly above the for line of the loop that fills the array. Note that this pragma works for for loops and not while loops, so if you wrote
the loop as a while loop, re-write it as a for loop. You will also need to add an include:



```
#include <omp.h>
```



To compile the program, you will need to add a new compiler option -fopenmp. If the source code is located in the file main.cpp, the command to compile will
be:



```
g++ -fopenmp main.cpp -o main
```



Note that using OpenMP on a Mac requires some setup because the default clang compiler on MacOS does not support OpenMP by default. Follow these
directions to set up a Mac.
```

The final part of this assignment is for the students to learn to use the chrono library to benchmark the parallel and sequential code. They should find that there is speedup

Figure 17.5: Part 2 of Intro to OpenMP assignment

13. There is another way to find out how many processors your computer has. You will need to include the `<thread>` library.

```
#include <thread>
```

The value `thread::hardware_concurrency()` stores the number of processors. Add a line of code to print this out and see if it agrees with the answer you and above.

14. The `#pragma omp parallel for` basically splits the iterations of the loop so that they will be done in parallel by different threads. A thread is like a mini-program. To get to see how this works first reduce the size of the array (and the number of iterations of the loop) to 50. We are going to add a print statement to the loop and we don't need to watch millions of iterations. Inside the loop add the line of code:

```
id = omp_get_thread_num();
```

The function `omp_get_thread_num()` returns the thread number that is executing an iteration of the loop.

Print out the `id` inside the loop. How many different numbers are printed out? Does this make sense given the number of processors?

15. Add some additional text to your `cout` statement such as

```
cout << "id: " << id;
```

Does the output seem all jumbled up? Why do you think that could be happening?

To fix this problem, we can change to using a different function called `printf()`, that is actually part of the C programming language. The equivalent of the above `cout` using `printf` is:

```
printf("id: %d", id);
```

The `%d` is a format specifier for an integer. Try this out and confirm that it fixes the problem.

16. Now compile and run the code with and without the `#pragma omp parallel for`. How many thread ids do you see in both cases?

17. The `#pragma omp parallel for` is separating the loop into different chunks so that each chunk can be done by a different processor simultaneously (in parallel). Which loop indexes are being handled by each thread? (Hint: print it out and see)

18. Sometimes we want the threads to each have their own copy of a variable and sometimes we want all the threads to share a copy. Are all the threads sharing the indexing variable for the loop, or do they each have their own copy in this case? To find out, print out the memory location where the indexing variable is stored. The format specifier for a memory address/pointer is `%p`, use this to see what the address is for the indexing variable inside the loop. You should find that each thread has its own memory address for the indexing variable if you have written the loop using something like `for (long long ind = 0; ind < SIZE; ind++)`. What do you think would happen if each thread/process had to share the same memory location for the indexing variable? (HINT: It is the same problem as what happened in #15 when all the threads were competing for a shared resource, printing to the same terminal window. When threads/processes are competing for a resource and it causes unpredictable outcomes depending on which thread or process gets there first, this is called a race condition and is generally considered undesirable)

19. Given the number of processors on your machine, how much faster do you think this task should be if we use all of the processors?

in the parallel version, but not as much as they predicted. In this section, students are encouraged to spend some time using AI tools to learn about the chrono library to figure out how to use it for this application.

Figure 17.6: Part 3 of Intro to OpenMP assignment

Part 3: Explore the chrono library and measure if the parallelization is producing any speedup in the code's runtime.

20. [AI] To actually measure how fast the code runs we need to take a look at another C++ library: `chrono`. Use AI to find out about how to use this library to measure the execution time of a piece of code. The `<chrono>` library has several different types of clocks, so make sure you have a good understanding of what the different clocks are. Justify your choice of clocks. (Maybe try an AI prompt like "Help me learn to use `<chrono>` to benchmark my code".)

21. Figure out how to time the block of code that parallelizes the loop. Run the program several times using the `#pragma omp parallel for` and without using it. Are the times consistent for the same conditions? If not, why? The parallel for loop should be faster than the non-parallel version. Does it come close to the speed up that you predicted above in question 19? If not, why not? (You can use AI to help answer this question.)

22. How much time did this assignment take? Any other comments on how to make this assignment better for future students?

TURN IN: Your answers to the questions above and the code that you wrote and also how long this assignment took and any feedback that will make it better for future students.

- How-To Guides

Algorithms

Implementation Details

- Instructional Material (Lectures, Videos, Assignments, Tests)
- How-To Guides

Experience and Teaching Tips

Data and Analysis

17.4.2 Array summing in parallel using OpenMP

Problem Description, Prerequisites, and Learning Outcomes

Problem Description Students are instructed to write a C++ program to add all of the elements of the array and then to parallelize that loop using `pragma omp parallel`

for. They will make the discovery that this does not work as desired and the accumulator (which is being shared by all the threads) is a shared resource that is experiencing a race condition. The next step is the Unplugged coin addition activity. **Prerequisites** The Array Creation OpenMP (Section 17.4.1) activity (The Unplugged Coin Addition (Section 17.3.2) was designed and used as a co-requisite. **Learning Outcomes**

- Students will learn to recognize a race condition.
- Students will learn to recognize a shared variable.
- Students will describe why a simple loop parallelization using `#pragma omp parallel` for may fail

Algorithms

Implementation Details

- **Instructional Material (Lectures, Videos, Assignments, Tests) Repository** with assignment and code: https://github.com/charlottegruner/CDE_R_CC_CS1/tree/main/SummingArrayOpenMP The first part of the assignment asks students to write and then parallelize code to sum the contents of an array. Since at this point, students only know how to parallelize a for loop that does not take into account shared variables (from the Array Creation in OpenMP from Section 17.4.1 assignment), they should discover that this will fail and ideally question why this code is not working as expected.

Figure 17.7: Part 3 of Intro to OpenMP assignment

Introduction to parallel computing and OpenMP in C++ Part 2: shared variables

This activity is a pair programming exercise. This means that one person is at the keyboard (the driver) and the other is the navigator, observing, reviewing, and offering advice. This activity introduces the concept of a race condition and shared variables.

1. Starting with the code from [part 1 \(populating the array\)](#), that you did last week, add a new `for` loop that traverses the array, adding up all the values in the array using an accumulator. Check to make sure you are getting the correct answer using the formula for the sum of numbers from 1 to N is given by $N(N+1)/2$ (although the way we have populated the array, we are actually doing the sum from $0 + 1 + 2 + \dots + N-1$. Be sure that your code works for an N of 20,000,000. (This should be a non-parallel program with no OpenMP).
2. Add the `#pragma omp parallel for` from Part1 to both loops in the program. Are you still getting the correct answer?
3. Change your array size N to a smaller value. Examine the accumulator variable inside the accumulation loop along with the thread id for each part of this loop. What is going on here?

TURN IN: Code that does this task in parallel and in serial. The serial code should work. Make sure you have implemented the formula to prove that it works and you get the correct value! Each block of code should also have timing information.

The intended follow-on activity is the Unplugged Coin Addition (see Section 17.3.2 activity that will illustrate that the naive approach to summing the array with a shared variable won't work in parallel. Ideally, students will realize that the code to sum the array contents can be parallelized if each of the threads is responsible for finding a partial sum of part of the array. The final lesson is a lecture that explains how to implement this idea using OpenMP. The key part of the code is shown in Figure 17.8 A video explaining the solution is at: <https://casper.yuja.com/V/Video?v=15996706&a=206700495>

- **How-To Guides**

Figure 17.8: Part 3 of Intro to OpenMP assignment

```
#pragma omp parallel //denotes the start of a parallel region
{
    long long int partialSum = 0; //each thread will have it's own copy

    #pragma omp for //use this inside a parallel region instead of "pragma omp parallel for"
    for (long long int j = 0; j < N; j++) {
        partialSum += array[j]; //each thread computes a local sum
    }

    #pragma omp critical //marks code that only one thread at a time can use
    {
        sum += partialSum;
    }
}
```

Experience and Teaching Tips

Data and Analysis

17.4.3 Greenfoot project

Problem Description, Prerequisites, and Learning Outcomes

Problem Description Greenfoot is a Java-based game development platform designed specifically for education. The PDC-related objective was to expose students to Event Driven Programming. Students were given two preliminary assignments that served as a guided introduction to Greenfoot. The final deliverable was a novel game program that the students completed in teams of 2 or 3. The materials used were adapted from the Knox materials as described in Section 1.4.2.

Prerequisites Although Greenfoot is in Java, these students who were in the first half of the second semester of using C++ (CS1 is also a C++ class at Casper College) were able to easily understand the Java used in Greenfoot.

Learning Outcomes

- Exposure to Java programming language for a non-Java CS program. Although there is currently no Java-based courses at Casper College, this activity was a gentle exposure to Java.
- Application of Object Oriented Programming. This project allowed students to see fundamental OOP concepts in action including the use of inheritance in classes.
- API Utilization. Students had to learn to use the Greenfoot framework.
- Experience with Event Driven Programming. This is the primary tie in with PDC concepts that prompted the use of this activity.
- Collaboration experience. Students worked with at least one other teammate to produce a final product.

Algorithms

Implementation Details

- **Instructional Material (Lectures, Videos, Assignments, Tests)** The Greenfoot material was used as a midterm project and was presented to students in 3 phases. The

first phase was just an exploration to make sure students could get the environment to work with their own equipment and included a starter project (hedgehog). The instructions for this first part are shown in Figure 17.9.

Figure 17.9: Part 3 of Intro to OpenMP assignment

COSC 2030 FA25 Midterm Project Preliminary Work 1: Intro to Greenfoot

I'd like to use Greenfoot, which is a Java API for building 2-D games. Even though it is in Java, I think you will find the syntax very much like C++. I have a few motivations for this project:

- Exposure to Java -- this will be a gentle exposure to the Java programming language if you haven't seen it before
- See some of the concepts in object oriented programming in action.
- Use an API to develop a game
- Write a game program that can be part of your computer science portfolio of work
- Have an experience in Event Driven Programming

Our first hurdle will be to get the greenfoot environment working on a computer that you can use. I have requested that it be installed on the classroom computers (I thought until this week that we'd be able to do it ourselves), but I'm not sure how long that will take.

I need to know if you are going to need an alternate computing environment for this project.

Please attempt to download greenfoot (<https://www.greenfoot.org/download>) and get it running. I've included a greenfoot starter project (Hedgehog.zip) that you can try to open up to see if it is working. In the Hedgehog directory, find the greenfoot.project file and double click to open up the greenfoot environment.

Let me know your status in this assignment.

In the second phase, students were instructed to view an introductory video and then attempt to analyze the hedgehog code and make some changes to the game. The instructions for the second phase of the project are shown in Figure 17.10.

Figure 17.10: Part 3 of Intro to OpenMP assignment

COSC 2030 FA25 Midterm Project Preliminary Work Part 2:

Watch this video which introduces Greenfoot and talks about the hedgehog project.

<https://casper.yuja.com/MSEntry.jsp?redirectURL=https%3A%2F%2Fcasper.yuja.com%2F%2FVideo%3Fv%3D14006674%26a%3D23690992>

Now that you have an idea of how Greenfoot works, add another actor to the hedgehog game.

Take a look at the ShootingGame and watch this video introduction:

<https://casper.yuja.com/MSEntry.jsp?redirectURL=https%3A%2F%2Fcasper.yuja.com%2F%2FVideo%3Fv%3D14006674%26a%3D23690992>

In the video, Jaimie (the presenter) says he would re-write the section that positions the 'birds' which are really butterflies so that they don't touch each other without having to place every single one. Figure out how to do this.

TURN IN: a video summary of your work.

The final phase of this project was a short, team project (suggested time 5 hours per teammate) to produce a game in the Greenfoot environment. The directions for this phase are shown in Figure ???. The directions are intentionally open ended for this project since I was also a new Greenfoot user and I did not know what to expect.

Figure 17.11: Part 3 of Intro to OpenMP assignment

COSC 2030 FA25 Midterm assignment

Spend at least 5 hours (per student) in Greenfoot to create something that will impress the rest of the class. This will be a program that you will complete with a partner.

TURN IN:

- A record of your time log for each partner.
- A video demonstration of your game and an explanation of the code. This could be done jointly with each partner contributing part of the video or separately with each partner talking about his/her contribution to the project.

https://github.com/charlottegruner/CDER_CC_CS2/tree/main/Greenfoot

• How-To Guides

Experience and Teaching Tips

This activity was very successful. Including this project at the CS2 level meant that the students could easily learn the Java they needed to know and all the teams were able to

create games that worked reasonably well and they were clearly having a lot of fun doing this project. In weeks 6-9 (in a 16 week semester) students formed two teams of 2 and one team of 3 and were instructed to spend at least 5 hours per student either simultaneously or asynchronously creating something that would impress the rest of the class. Students were instructed to turn in time-logs and also give a presentation on their final product. Most students spent more than 5 hours on the project and were able to make a lot of progress in a fairly short time. The students also enjoyed showing off their projects.

Data and Analysis

17.4.4 Parallel QuickSort

Problem Description, Prerequisites, and Learning Outcomes

Problem Description These materials are adapted from April Crockett's materials for CS2 (LAB 4 - Sort Algorithms, Section ??) as in in-class team project. The students are provided with code that implements a serial version of QuickSort, then asked to discuss with their team whether or not QuickSort is a good candidate for parallelization. The materials gave them some hints about proceeding with the parallelization. **Prerequisites** Students should have worked with OpenMP, chrono, and also have experience with the QuickSort algorithm. **Learning Outcomes**

- Students will think critically about the QuickSort algorithm implementation in C++.
- Students will learn about and apply the PDC concept of task parallelization.
- Students will learn about and apply new features of OpenMP.
- Students will practice collaborative problem solving.

Algorithms

Implementation Details

- **Instructional Material (Lectures, Videos, Assignments, Tests)**

Students are given a file of movie titles and starter code that reads the movie titles from movies.txt into a string vector and uses an implementation of the QuickSort algorithm in serial. In the first part of the assignment (instructions shown in Figure 17.12, which the live class did as an in class activity in groups of 2-3 students, the students are instructed to take a look at the code, make sure it works and that they understand it and also add in the necessary code that uses the chrono library to benchmark the code with the given movies.txt file. At the end of part 1, students are instructed to discuss with their group the question of whether the recursive calls to the serialQuickSort function could happen in parallel.

In the second part of the activity (shown in Figure 17.13), student are guided through the process of parallelization of the code. This was a new assignment, so I asked

Figure 17.12: Part 3 of Intro to OpenMP assignment

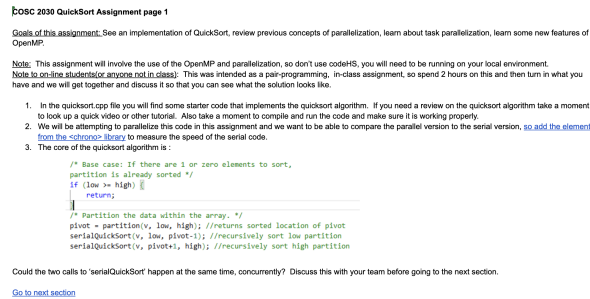
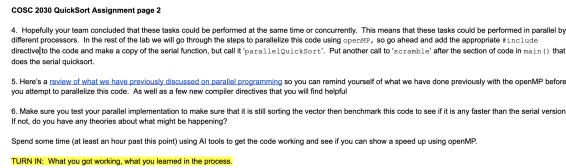


Figure 17.13: Part 3 of Intro to OpenMP assignment



students to spend time on it and report progress rather than expecting them to be able to get to the answer on their own.

https://github.com/charlottegruner/CDER_CC_CS2/tree/main/QuickSortAssignment

• How-To Guides

Experience and Teaching Tips

I normally don't do any new material on the Tuesday before Thanksgiving, but I realized that this might be the ideal time to talk about the parallelization of at least one sorting algorithm and would additionally give me one more chance to review the PDC content we had already covered and introduce one more new piece of PDC content. Since it was the day before Thanksgiving holiday, there were not too many students (3 in person, 1 on-line). I briefly introduced the activity and then let them take a look at the QuickSort code with their partner. We then had a group discussion about whether or not this algorithm was a good candidate for parallelization. I gave them some hints on how to use OpenMP to parallelize the code and let them use AI tools to get additional help. We had group discussion at regular intervals as they were working so they could share discoveries. We kind of ended up using a "best ball" approach in that during the group discussions in which I guided the discussion toward adopting the approach taken by the group that was going in the most promising at each junction that we stopped to discuss progress. In the span of the 2-hour class, they were able to basically re-create the code that April had written for parallel QuickSort with help from me and AI. This worked very well for a live class, but I'm not sure how well it would work for an on-line asynchronous student.

Data and Analysis

17.5 Course-wide Pre and Post Course Surveys

Course-wide pre and post surveys were given to students in the Fall 2024 semester (No PDC interventions) and Spring 2025 (PDC interventions as described) semesters. As mentioned previously, these were not large classes. The Fall 2024 semester had 16 students. Of this population, 9 students completed the pre-course survey and 7 students completed both the pre-course survey and the post-course survey and also gave consent for data to be used for this study. The Fall 2025 semester had 6 students. In this class, 4 students completed the pre-course survey and 2 both pre- and post-course surveys and gave consent.

The following plots show the individual student's response to the same question in both the pre-course survey and the post course survey for some of the questions asked on the survey. The student responses in blue are from the Spring 2025 class that did not have any PDC interventions, the responses in orange are from the Spring 2026 class. When more than one student gave the same pre, post coordinate, the size of the dot is increased proportionally. The PDC intervention dots are slightly offset by .1 from the non intervention plots to allow for the overlapping coordinates not to have overlapping dots. For each question, the student was asked to rate on a Likert Scale of 1 (strongly disagree) to 7 (strongly agree) their opinion on each question.

Questions that asked students about their level of interest in computing in general and parallel and distributed computing Figure 17.14 student interest in computing in general, Figure 17.15 shows student interest in PDC topics. It is clear that all of these students have high and sustained interest levels in computing. The non-intervention group seems to share this interest in parallel distributed computing. One student in the intervention group increased from a ranking of 2 in the pre-course survey to a 4 in the post-course survey. The other student in the intervention group dropped from 6 to 3. This same student also dropped interest in computing from 7 in the pre-course survey to 6 in the post course survey perhaps reflecting an overall decrease in interest.

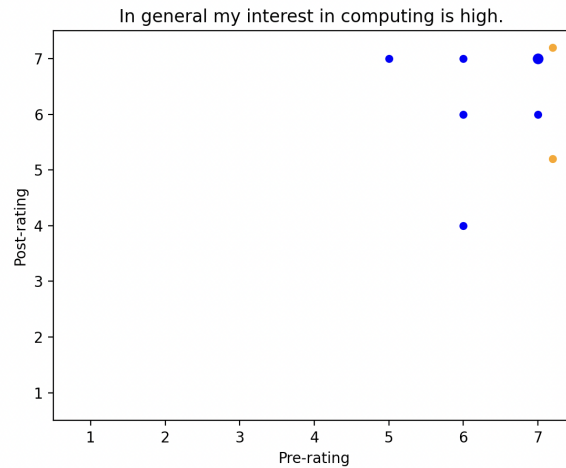
Comparison between topics covered in both classes vs PDC intervention topics Several questions asked students to reflect upon their "experience" with different topics. These plots show the student responses for topics covered in both non-intervention and pdc intervention and classes (Lists/Linked Lists, Trees, Classes), topics covered in Fall 2025 only (event handling, parallel processing), and one topic that was not covered in either class (distributed computing). In general the plots asking about have a similar pattern of students rating themselves the same or higher at the end of the class.

Questions that asked students about their perceived level of understanding.

17.6 Other Activities and Ideas

[Alternative ways to apply these concepts, etc..]

Figure 17.14: Student responses to the question "In general, my interest in computing is high". Blue dots are student responses from Spring 2025 in which there was no PDC intervention. Orange dots are student responses from Spring 2026 in which there were several PDC interventions.



17.7 Conclusion

17.8 Other Activities and Ideas

[Alternative ways to apply these concepts, etc..]

17.9 Conclusion

Figure 17.15: Student responses to the question "My interest in parallel and distributed computing is high". Blue dots are student responses from Spring 2025 in which there was no PDC intervention. Orange dots are student responses from Spring 2026 in which there were several PDC interventions.

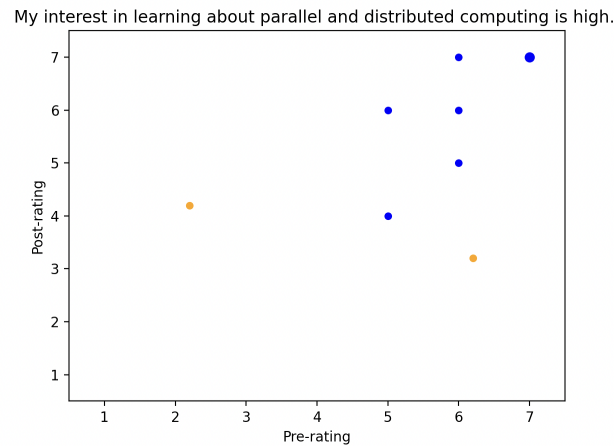


Figure 17.16: Student responses to the question "Please rate your experience with using Queues in your programs". Blue dots are student responses from Fall 2024 in which there was no PDC intervention. Orange dots are student responses from Fall 2025 in which there were several PDC interventions.

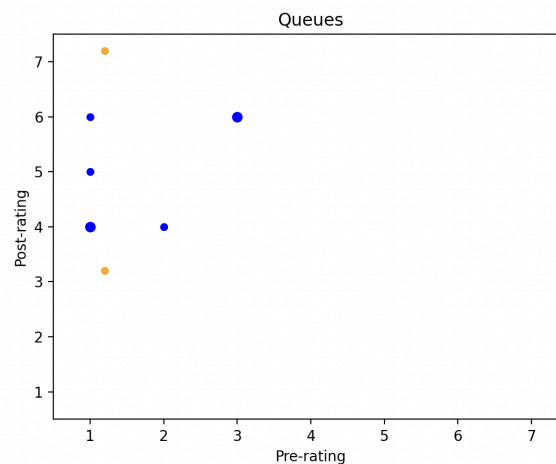


Figure 17.17: Student responses to the question "Please rate your experience with using classes & objects in your programs". Blue dots are student responses from Fall 2024 in which there was no PDC intervention. Orange dots are student responses from Fall 2025 in which there were several PDC interventions.

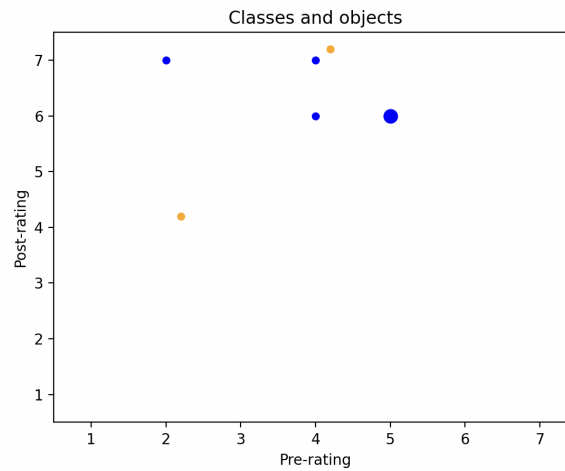


Figure 17.18: Student responses to the question "Please rate your experience with Big O and algorithmic complexity". Blue dots are student responses from Fall 2024 in which there was no PDC intervention. Orange dots are student responses from Fall 2025 in which there were several PDC interventions.

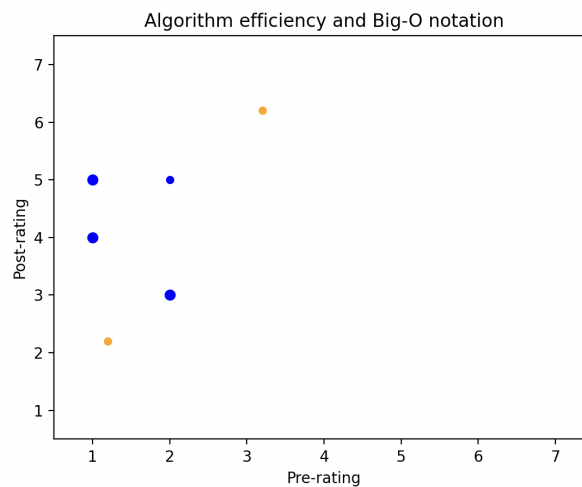


Figure 17.19: Student responses to the question "Please rate your experience using Trees in your programs". Blue dots are student responses from Fall 2024 in which there was no PDC intervention. Orange dots are student responses from Fall 2025 in which there were several PDC interventions.

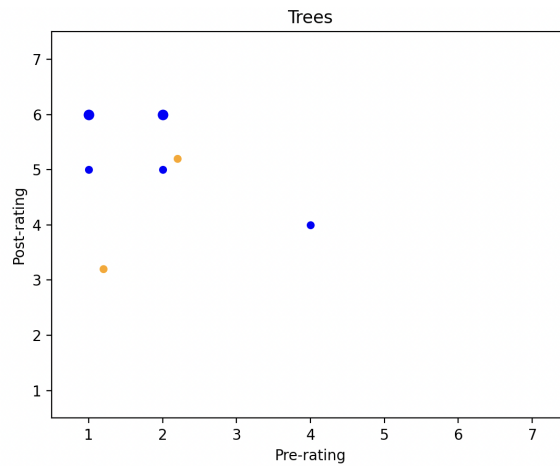


Figure 17.20: Student responses to the question "Please rate your experience using B-Trees in your programs". Blue dots are student responses from Fall 2024 in which there was no PDC intervention. Orange dots are student responses from Fall 2025 in which there were several PDC interventions.

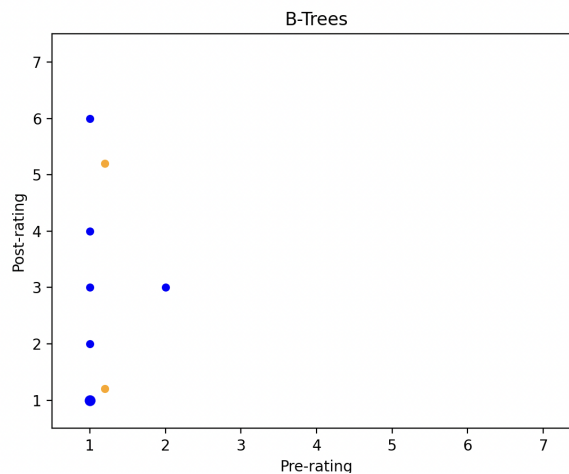


Figure 17.21: Student responses to the question "Please rate your experience using Lists/Linked Lists in your programs". Blue dots are student responses from Fall 2024 in which there was no PDC intervention. Orange dots are student responses from Fall 2025 in which there were several PDC interventions.

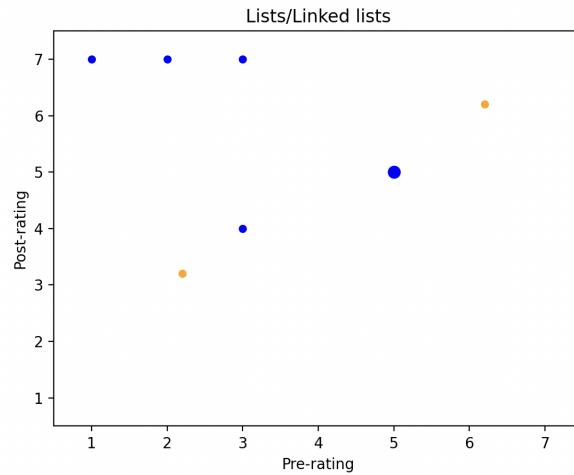
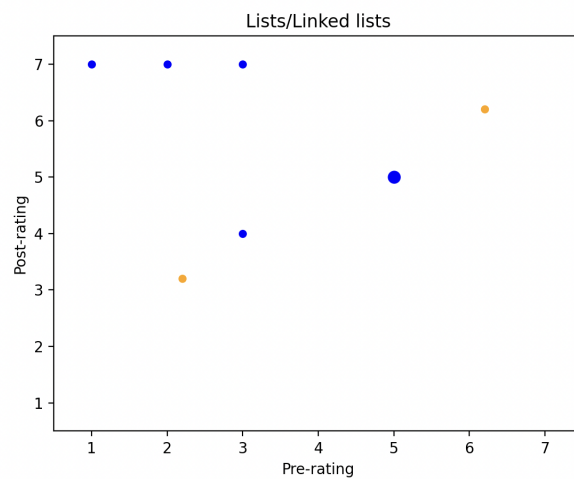


Figure 17.22: Student responses to the question "Please rate your experience using in your programs". Blue dots are student responses from Fall 2024 in which there was no PDC intervention. Orange dots are student responses from Fall 2025 in which there were several PDC interventions.



Bibliography

Appendix

|You must include these four specific subsections (in addition to anything else):|

Github Link to Instructional Material

`https://github.com/...`

Detailed Pre and Post Syllabus

Organization of Material

|Explain the folder structure of your Github repository.|

Survey and Other Anonymized Data and Analysis

|Include student feedback, learning data, or implementation analysis.|